

Design documentation.

Language: C; RTOS: Zephyr 4.4.99

The decision was between Nordic nRF9161/nRF9160/nRF9151 and I decided on nRF9151 and its main advantages are:

- the newest one and has newer features (class 5 TX power – less power demanding)
- has smaller peak power consumption compared to two other ones
- it's the cheapest of the three
- probably will have the longest support and updates

Newest device down side could be bugs that will be fixed through its life cycle.

I didn't consider devices that didn't have LTE-M built in to make the device more power efficient and easier to develop, additionally Nordic has good Zephyr support.

For data protection in the case of losing power or failed data upload, we can have second ring buffer that saves larger amount of air quality data and writes it in NVM/FLASH memory, this helps with FLASH memory write cycles.

1. Protocol choice: I chose CoAP+DTLS: (emphasis on battery life)

- + Uses UDP which needs less connection setup, so less time needed for modem to be up
- + Built for small telemetry data sending
- + DTLS can reduce key renegotiation overhead after every upload event
- Less commonly used and more difficult to implement than others
- + Keep the design simple

2. Power strategy

When threads are not active device should enter low power mode, MCU remains in IDLE, modem turns off, RTC can stay on. Documentation mentions that device can use few uA of current with correct setup, that helps with longer battery life. This device supports power class 5 /20dBm/, if network connection supports it power consumption can be reduced. If modem fails connection to network, fails uploading data or has any other error, device will retry multiple times and if it still fails it returns to low power mode and retries on the next scheduled upload.

3. Security basics (CRA awareness)

1. DTLS provides encryption and server authentication for CoAP telemetry data. If any part of DTLS fails, data doesn't get sent.
2. No sensitive data should be stored in source code, git or anywhere that can be public at some point. Private keys, tokens, passwords should all be stored in protected storage, like protected flash area.
3. Device would use Zephyr's MCUboot and new firmware would be downloaded over secure LTE-M connection. New firmware would be stored in flash space (two fw images at the same time) and MCUboot would verify it before using. If new firmware fails at any point it can rollback to previous firmware.

4. Remote troubleshooting

1. Firmware already uses structured Zephyr log messages (info, warning, error) for most firmware functions.
2. Logs should be stored in NVM/FLASH for easy access if the system fails. To reduce wear and tear for NVM/FLASH and increase its lifespan we should pack data in bigger blocks and do writes less often.

3. The code can support an option to either upload log data automatically if device crashes or a technician requests them from the cloud.

If the device is nonresponsive then physical access can be added, for example UART through USB, to enable it when needed physical pin state change can be added in code to activate UART and disable sleep.

5. If I had more time I would implement:

1. Diagnostic logs stored in FLASH/NVM.
2. Current k_msgq buffer drops newest read sensor sample if it is full. I would change code so that it removes the oldest sample in the buffer and adds the newest sample.
3. Split code into more source files (sensor.c, modem.c...) and add header files for better structure.